



Distributed Systems

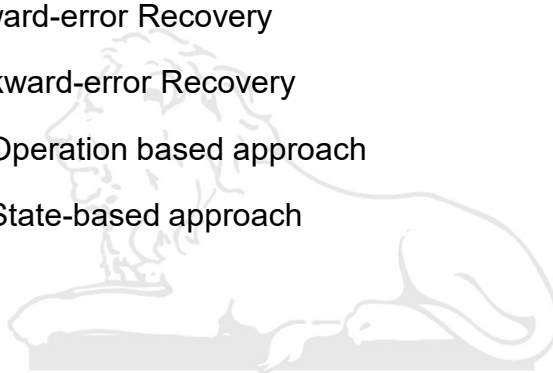
Fault Tolerance & Recovery



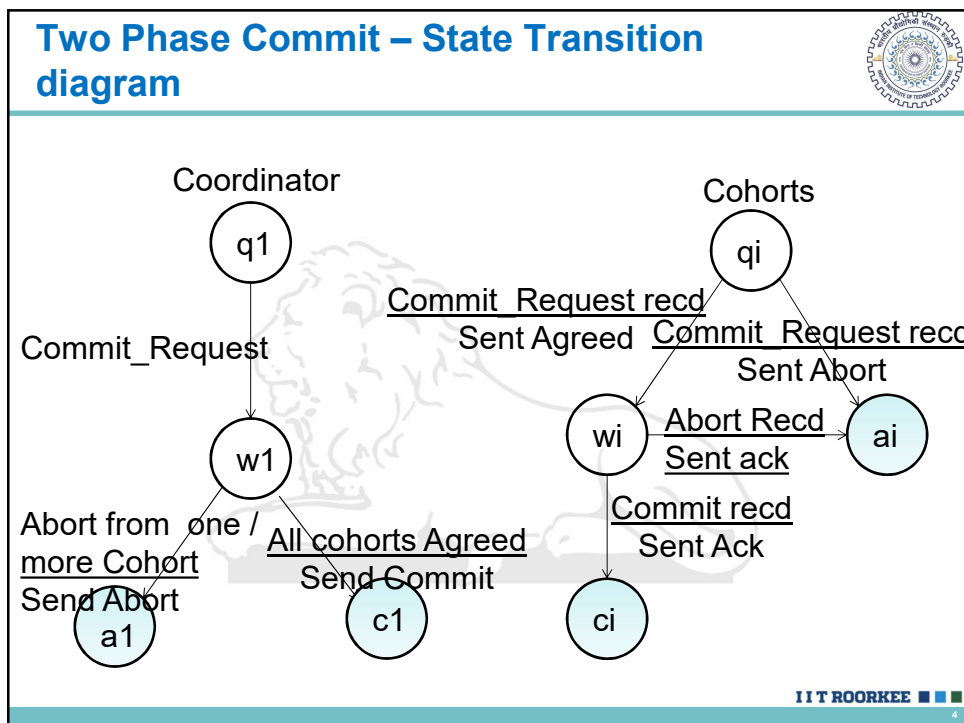
Recap



- Error recovery
 - Forward-error Recovery
 - Backward-error Recovery
 - Operation based approach
 - State-based approach
- 2PC



- Non-blocking protocols possible; ex., **Three-Phase Commit protocol**
- **Non-blocking protocols**
 - Operational sites should agree on the outcome of the transaction
 - Failed sites should all reach the same conclusion upon recovery
- **Independent recovery**
 - If recovering sites can decide the final outcome based on their local state

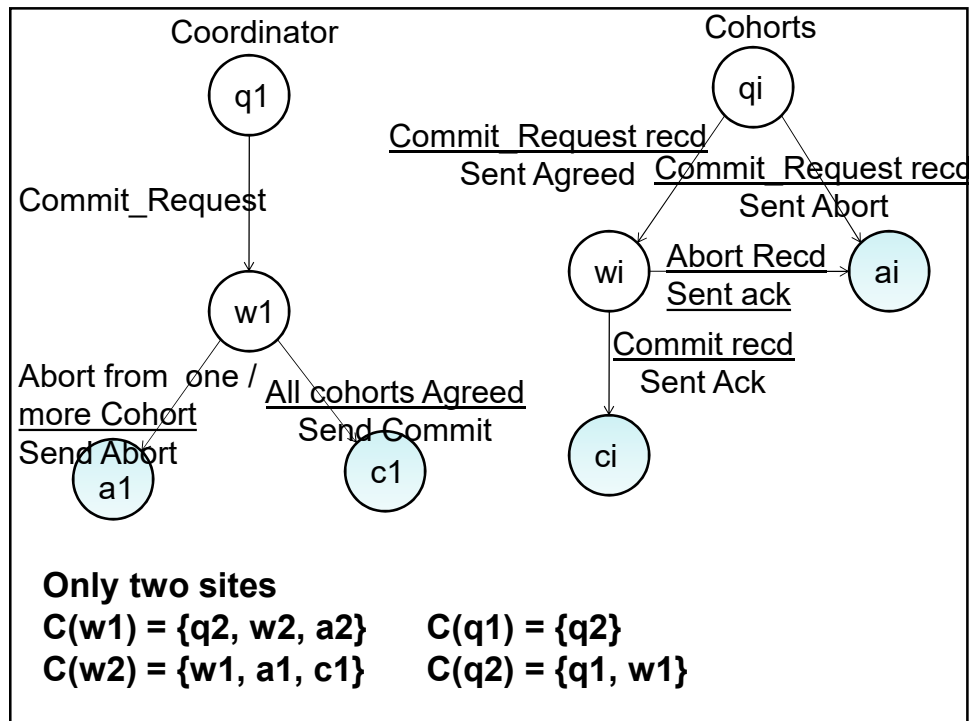
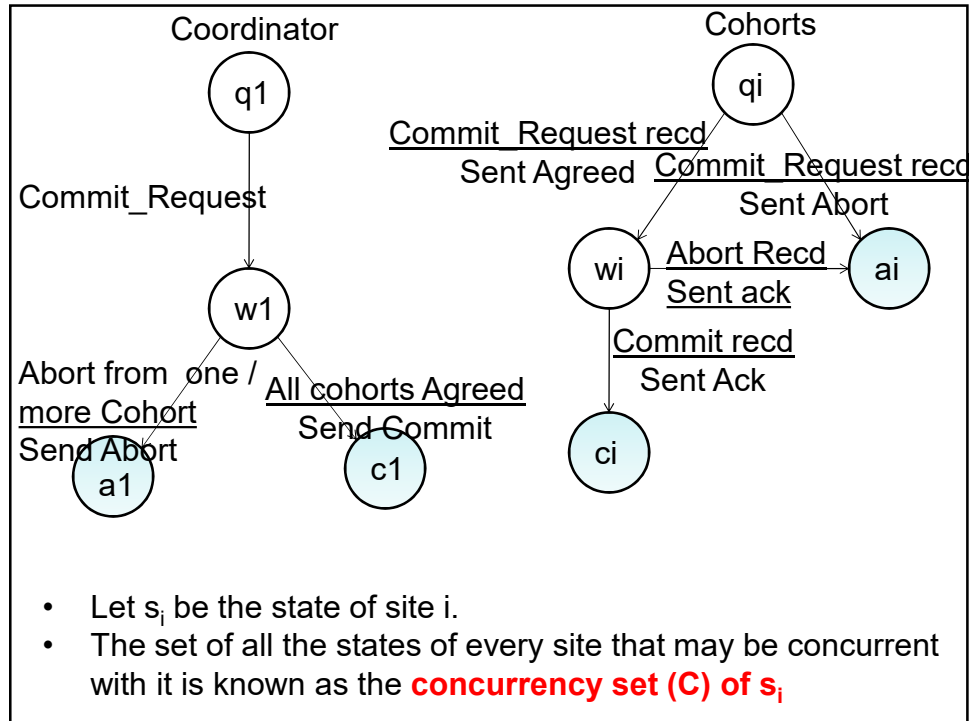


Assumptions

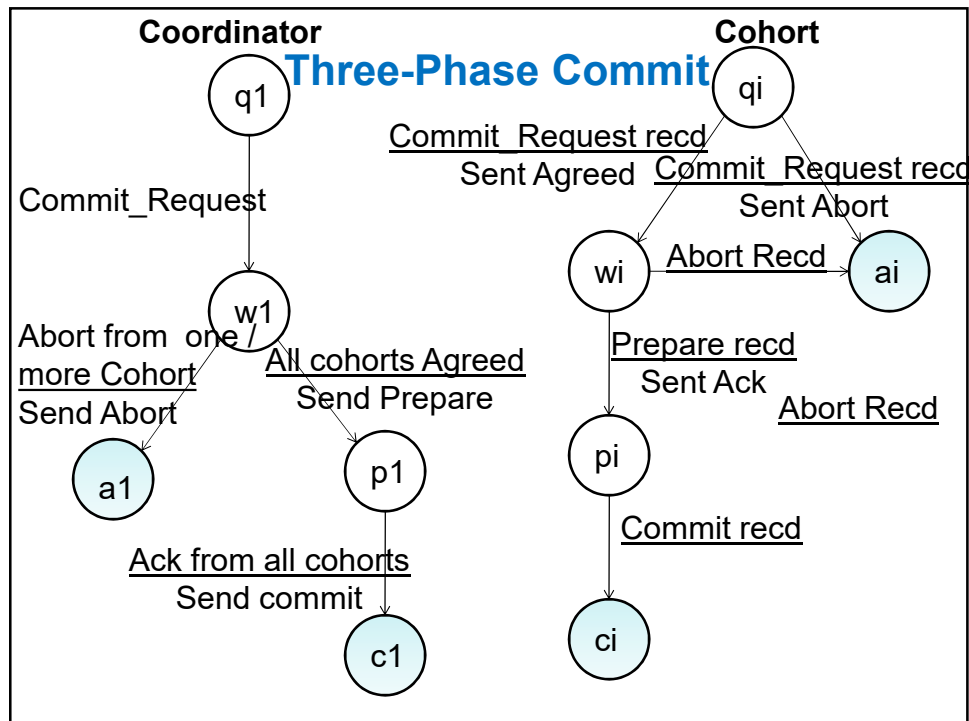
- The network is reliable and point-to-point communication is possible between any two operational sites
- The network can detect the failure of a site and report it to the site trying to communicate with failed site

Synchronous (within one state transition) Protocols

- One site never leads another site by more than one state transition during the execution of the protocol
- Let t_k be the total number of state transitions at site k
 For all i, j , $|t_i - t_j| \leq 1$
 where $1 \leq i, j \leq n$;
 n is the number of sites;
 A state transition occurs in the coordinator / cohort whenever it receives and / or sends messages
- Does Two phase commit protocol satisfy this condition?



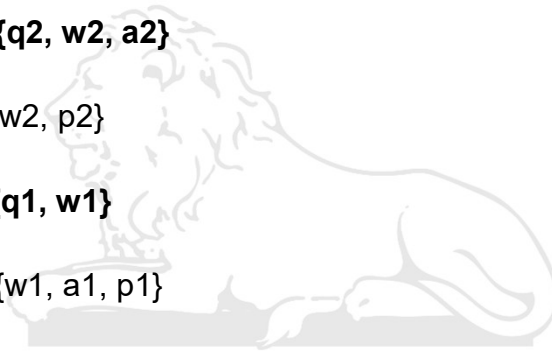
If a protocol contains a local state of a site with both abort and commit states in its concurrency set, then under independent recovery conditions it is **NOT** resilient to an **arbitrary single failure**



Three-Phase Commit



- $C(q1) = \{q2\}$
- $C(w1) = \{q2, w2, a2\}$
- $C(p1) = \{w2, p2\}$
- $C(q2) = \{q1, w1\}$
- $C(w2) = \{w1, a1, p1\}$
- $C(p2) = \{p1, c1\}$



References



- Singhal M and Shivaratri N, Advanced Concepts in Operating Systems, Tata McGraw Hill
- Dale Skeen, Nonblocking Commit Protocols ACM SIGMOD International conference on Management of Data 1981



Sender Set: Let s be a state of a site, and let M be the set of messages that are received in state s , the sender set for s

$$S(s) = \{t \mid t \text{ sends } m \text{ and } m \in M\}$$

A local state is called **committable** if occupancy of that state by any site implies that all sites have voted yes on committing the transaction.

Failure Transitions

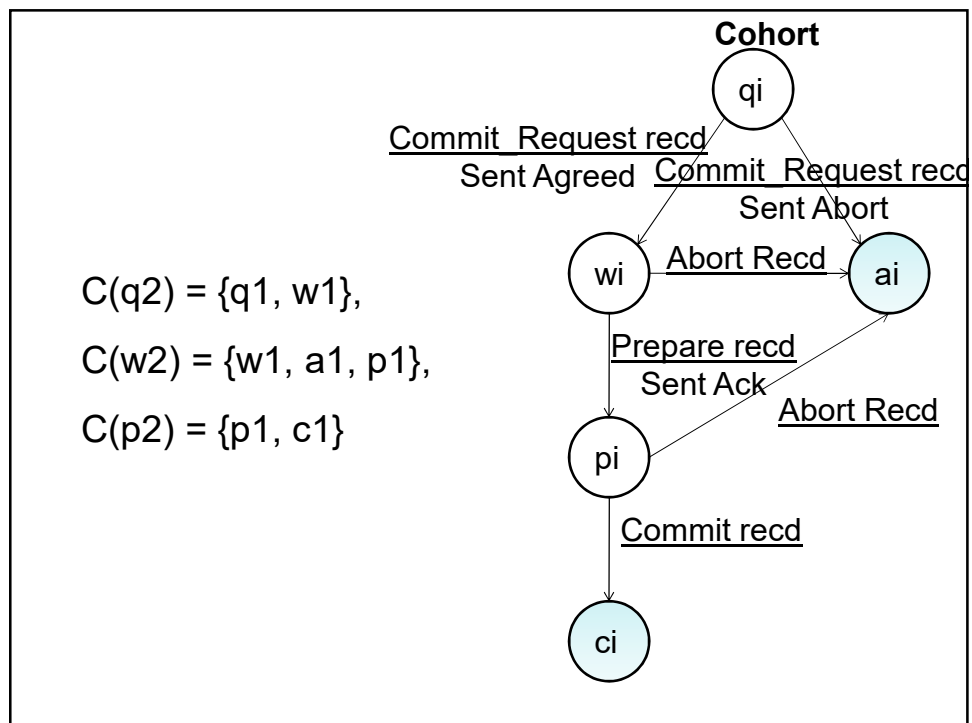
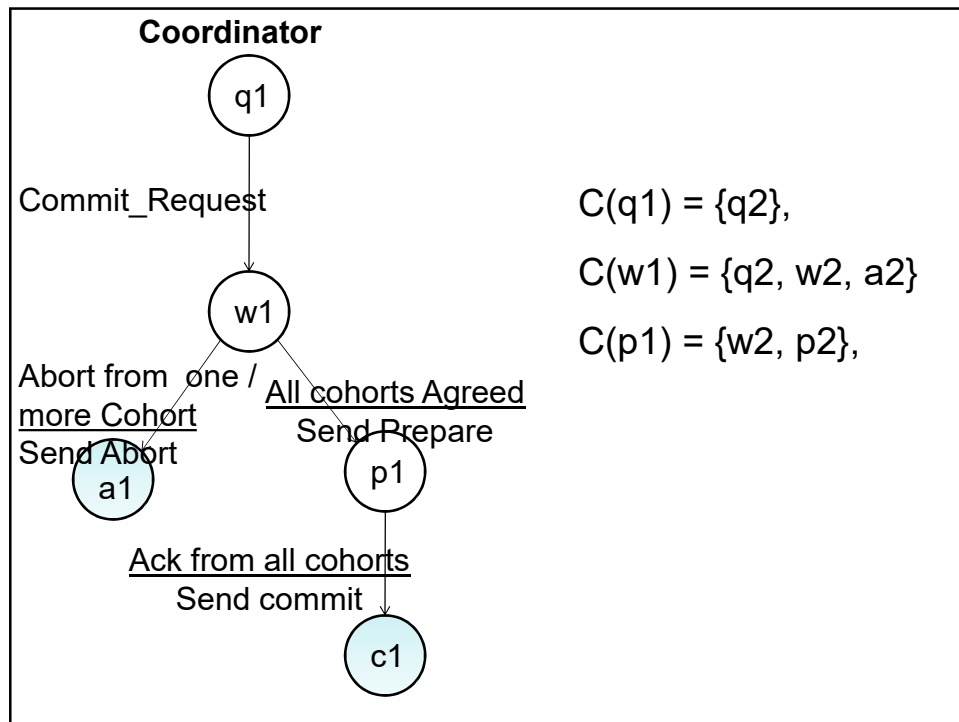
Occurs at the failed site at the instant it fails or immediately after it recovers

Rule: For every non-final (and non-committable) state s , if $C(s)$ contains commit, then assign a failure transition to commit else to abort

Timeout Transitions

What an operational site does in the event of failure of other site (i is waiting for a message from j ; j has failed)

Rule: For each non-final state s , if j is in $S(s)$ and j has a failure transition to a commit (abort) state, then assign a timeout transition from s to commit (abort).



Multiple Site Failure and Network Partitioning



- There exists no protocol using independent recovery that is resilient to arbitrary failures by two sites
- There exists no protocol resilient to network partitioning when messages are lost
- There exists no protocol resilient to multiple network partitioning

Checkpoint and Rollback Recovery



- It is a form of backward error recovery

Checkpoint :

- **local checkpoint** – local state of a process saved in **stable storage** for possible rollback on a fault
- **global checkpoint** – collection of local checkpoints, one from each process

Consistent and **Strongly Consistent** Global Checkpoint
– similar to consistent and strongly consistent global state respectively (Also called “recovery line”)

$\text{send}(m_{ij}) \in \text{LC}_i$ iff
 $\text{time}(\text{send}(m_{ij})) < \text{time}(\text{LC}_i)$

$\text{rec}(m_{ij}) \in \text{LC}_j$ iff
 $\text{time}(\text{rec}(m_{ij})) < \text{time}(\text{LC}_j)$

$\text{transit}(\text{LC}_i, \text{LC}_j) = \{ m_{ij} \mid \text{send}(m_{ij}) \in \text{LC}_i \wedge \text{rec}(m_{ij}) \notin \text{LC}_j \}$

$\text{inconsistent}(\text{LC}_i, \text{LC}_j) = \{ m_{ij} \mid \text{send}(m_{ij}) \notin \text{LC}_i \wedge \text{rec}(m_{ij}) \in \text{LC}_j \}$

Global Checkpoint: collection of local checkpoints
 $\text{GC} = \{\text{LC}_1, \text{LC}_2, \dots, \text{LC}_n\}$

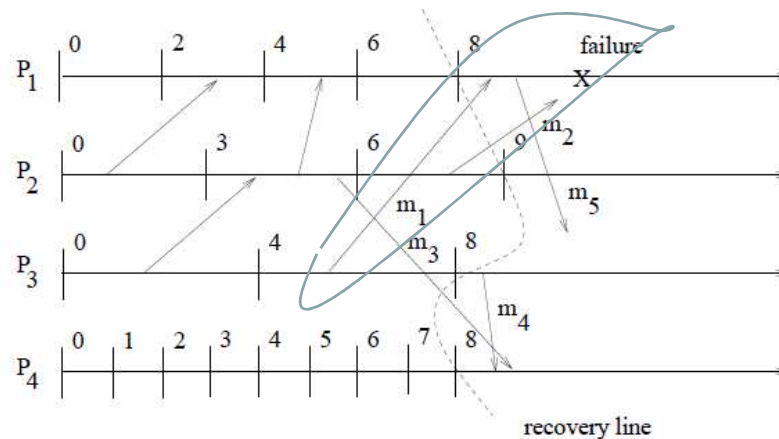
GC is consistent iff
 for all $i, j, 1 \leq i, j \leq n$,
 $\text{inconsistent}(\text{LC}_i, \text{LC}_j) = \emptyset$

GC is transitless iff
 for all $i, j, 1 \leq i, j \leq n$,
 $\text{transit}(\text{LC}_i, \text{LC}_j) = \emptyset$

GC is strongly consistent if it is consistent and transitless
 - corresponds to a consistent global state in which all channels are empty

In-Transit Messages - a message *that has been sent but not yet received* (m1, m2)

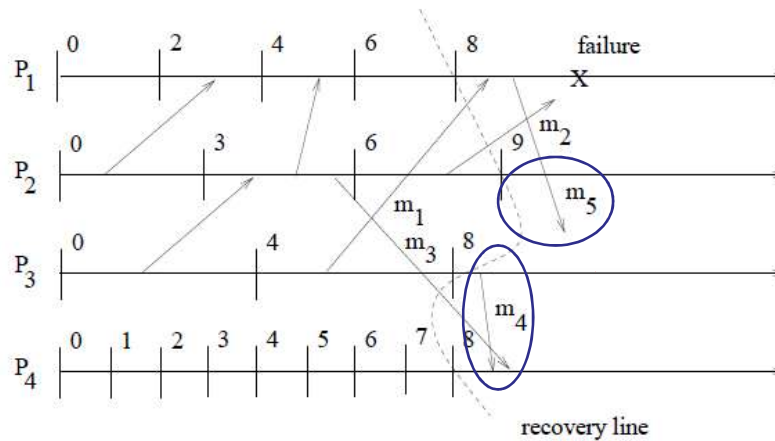
Lost message – Messages whose send is not undone but receive is undone due to rollback are called lost messages (m1)



- Handling of In-transit messages depends on the type of channel
 - Reliable channels
 - Lossy channel
- Is lost messages a problem??
 - not if unreliable channels assumed (since messages can be lost anyway)
 - if reliable channels assumed, need to handle this properly! Cannot lose messages !
 - We will assume unreliable channels for simplicity

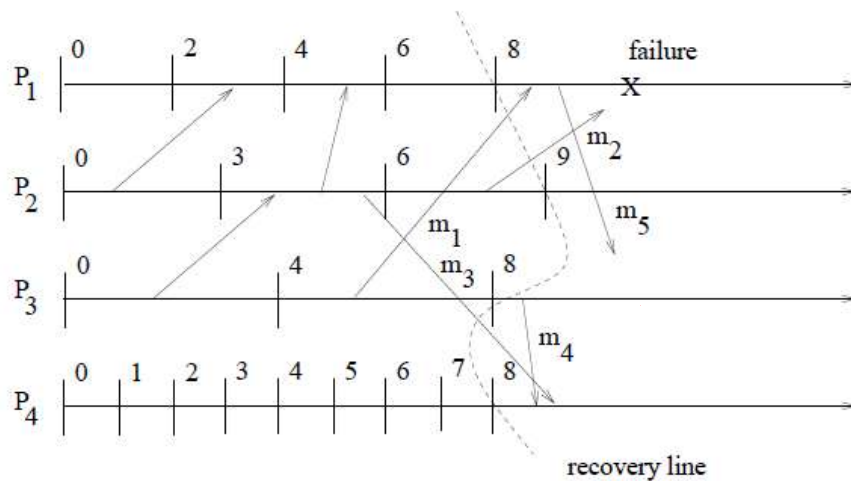
Duplicate messages: Arise due to message logging and replying during process recovery (m4, m5)

Delayed Messages – Receive of the message is not recorded because the receiving process was either down or the message arrived after the rollback (m2 and m5)

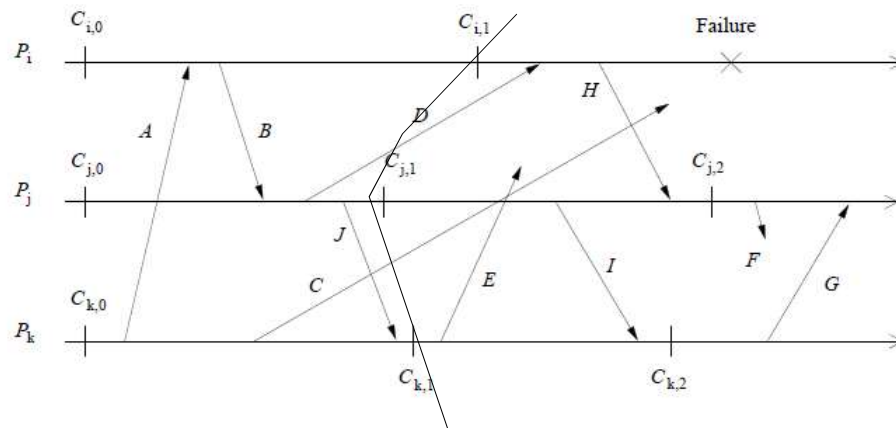


Orphan message – a message whose receive is recorded in some local checkpoint of a global checkpoint but send is not recorded in any local checkpoint in that global cp (m5)

A consistent global checkpoint cannot have an orphan msg



Issues in failure recovery



Normal, Vanished and other messages